Bài toán quản lý cây thư mục trong Windows

Mục đích
Cho sinh viên làm quen với cách ứng dụng của cấu trúc cây tổng quát trong một ứng dụng thực tế + Các thao tác duyệt cây + Thêm xóa, nút + Di chuyển nút

Mô tả
Quản lý các thư mục trong Windows hoặc linux dùng cấu trúc cây tổng quát với 1 nút được định nghĩa như sau.

A. Trong windows

Do windows có sử dụng các ký tự unicode nên thay vì dùng char chúng ta sẽ dùng WCHAR. Các bạn cần thay các hàm phù hợp (tham khảo thêm trên google).

```
#include <windows.h>
#include <wchar.h>
```

Để quản lý 1 cây thư mục ta có thể khai bao cấu trúc cây tổng quát như sau

Định nghĩa 1 nút

```
typedef struct TreeNode {
    WCHAR name[MAX_PATH];              // Tên của tệp hoặc thư mục
    WCHAR path[MAX_PATH];              // Đường dẫn đầy đủ (chỉ áp dụng cho gốc),
                                       //nút con sẽ để trống
    BOOL isDirectory;                  // TRUE nếu là thư mục, FALSE nếu là tệp
    struct TreeNode* parent;           // Con trỏ tới nút cha
    struct TreeNode* firstChild;       // Con trỏ tới nút con đầu tiên
    struct TreeNode* nextSibling;      // Con trỏ tới nút anh em tiếp theo
} TreeNode;
```

Ví dụ

Hàm thêm mới 1 nút trên cây

```
TreeNode* createNode(const WCHAR* name, const WCHAR* path, BOOL isDirectory) {
    TreeNode* node = (TreeNode*)malloc(sizeof(TreeNode));
    wcscpy(node->name, name);
    wcscpy(node->path, path);
    node->isDirectory = isDirectory;
    node->parent = NULL;
    node->firstChild = NULL;
    node->nextSibling = NULL;
    return node;
}
```

Để in ra đường dẫn tới thư mục hiện tại ta sẽ dùng hàm hệ thống `GetCurrentDirectory` như sau

```
    WCHAR root_directory[MAX_PATH];
    DWORD dwRet;

    // Lấy thư mục hiện tại
    dwRet = GetCurrentDirectory(MAX_PATH, root_directory);

    if (dwRet == 0 || dwRet > MAX_PATH) {
        // Xảy ra lỗi
        printf("GetCurrentDirectory failed (%lu)\n", GetLastError());
        return 1;
    }

    // In ra thư mục hiện tại
    wprintf(L"Current directory: %s\n", root_directory);
```

Hàm để duyệt và đọc các file/ thư mục trên windows là

- `FindFirstFile`: đọc thông tin file/thu mục con đầu tiên của thư mục hiện tại
- `FindNextFile:` Đọc thông tin file/thư mục tiếp theo

```
void list_files_and_folders(const WCHAR* root_dir, int indent) {
    WIN32_FIND_DATA find_file_data;
    HANDLE h_find = INVALID_HANDLE_VALUE;

    WCHAR path[1024];

    // Thêm dấu * để duyệt mọi file và thư mục
    swprintf(path, sizeof(path), L"%s\\*", root_dir);

    h_find = FindFirstFile(path, &find_file_data);

    if (h_find == INVALID_HANDLE_VALUE) {
        printf("FindFirstFile failed (%d)\n", GetLastError());
        return;
    }
    int check;

    do {
        const WCHAR* name = find_file_data.cFileName;

        if (wcscmp(name, L".") == 0 || wcscmp(name, L"..") == 0) {
            continue; // Bỏ qua thư mục hiện tại và thư mục cha
        }
        check = 0;
        if (find_file_data.dwFileAttributes & FILE_ATTRIBUTE_DIRECTORY) {
            printf("+");
            check = 1;
        }

        for (int i = check; i <= indent; i++) {
            printf("-");
        }

        wprintf(L"%s\n", name);

        if (find_file_data.dwFileAttributes & FILE_ATTRIBUTE_DIRECTORY) {
```

```
            WCHAR sub_dir[1024];
            swprintf(sub_dir, sizeof(sub_dir), L"%s\\%s", root_dir, name);
            list_files_and_folders(sub_dir, indent + 1); // Gọi đệ quy cho thư mục con
        }

    } while (FindNextFile(h_find, &find_file_data) != 0);

    FindClose(h_find);
}
```

Để tạo thư mục mới ta có thể dùng hàm `CreateDirectory`

```
    LPCWSTR directoryName = L"new_directory";

    // Tạo thư mục mới
    if (CreateDirectory(directoryName, NULL) || GetLastError() == ERROR_ALREADY_EXISTS) {
        wprintf(L"Directory '%s' created successfully or already exists.\n", directoryName);
    }
    else {
        wprintf(L"Failed to create directory '%s'. Error: %lu\n", directoryName,
GetLastError());
    }
```

B. Trong Unix/Linux

Kiểu char trong Linux/Unix hỗ trợ UTF-8 nên ta có thể dùng trực tiếp để lưu trữ tên file/thư mục có các ký tự Unicode.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef int BOOL;
#define TRUE 1
#define FALSE 0

#define MAX_NAME_LEN 256
#define MAX_PATH_LEN 1024

// Định nghĩa cấu trúc một nút trên cây tổng quát
typedef struct TreeNode {
    char name[MAX_NAME_LEN];        // Tên của thư mục hoặc tệp
    char path[MAX_PATH_LEN];        // Đường dẫn đầy đủ (chỉ áp dụng cho gốc),
                                    //nút con sẽ để trống
    int isDirectory;                // 1 nếu là thư mục, 0 nếu là tệp
    struct TreeNode* parent;        // Con trỏ tới nút cha
    struct TreeNode* firstChild;    // Con trỏ tới nút con đầu tiên
    struct TreeNode* nextSibling;   // Con trỏ tới nút anh em tiếp theo
} TreeNode;
```

Hàm thêm 1 nút mới

```
TreeNode* createNode(const char* name, const char* path, int isDirectory) {
    TreeNode* newNode = (TreeNode*)malloc(sizeof(TreeNode));
```

```
    if (newNode) {
        strncpy(newNode->name, name, MAX_NAME_LEN);
        strncpy(newNode->path, path, MAX_PATH_LEN);
        newNode->isDirectory = isDirectory;
        newNode->parent = NULL;
        newNode->firstChild = NULL;
        newNode->nextSibling = NULL;
    }
    return newNode;
}
```

Để in ra đường dẫn đến thư mục hiện tại ta dùng như sau

```
char root_directory[PATH_MAX];

// Lấy thư mục hiện tại
if (getcwd(root_directory, sizeof(root_directory)) != NULL) {
    // In ra thư mục hiện tại
    printf("Current directory: %s\n", root_directory);
}
else {
    // Xảy ra lỗi
    perror("getcwd() error");
    return 1;
}
```

Hàm đọc thư mục con và các file trong thư mục hiện tại cần dùng các hàm thư viện dirent.h

- **`opendir()`**: Mở thư mục gốc.
- **`readdir()`**: Đọc từng mục (thư mục hoặc tệp tin) trong thư mục.
- **`closedir()`**: Đóng thư mục sau khi duyệt xong.

```
void list_files_and_folders(const char* root_dir, int indent) {
    DIR* dir;
    struct dirent* entry;

    if ((dir = opendir(root_dir)) == NULL) {
        perror("opendir() error");
        return;
    }

    while ((entry = readdir(dir)) != NULL) {
        if (strcmp(entry->d_name, ".") == 0 || strcmp(entry->d_name, "..") == 0) {
            continue;
        }

        for (int i = 0; i < indent; i++) {
            printf("  ");
        }

        printf("%s\n", entry->d_name);

        if (entry->d_type == DT_DIR) {
            char path[1024];
            snprintf(path, sizeof(path), "%s/%s", root_dir, entry->d_name);
            list_files_and_folders(path, indent + 1);  // Gọi đệ quy cho thư mục con
        }
```

```
    }

    closedir(dir);
}
```

Để tạo thư mục mới ra dùng

```
const char* directoryName = "new_directory_2\\sub_directory";

// Tạo thư mục với quyền truy cập 0755 (rwxr-xr-x)
int status = mkdir(directoryName, 0755);
//status = mkdir("sample_dir", 0755);
//status = mkdir("new_directory_2\\sub_directory2", 0755);
if (status == 0) {
    printf("Directory '%s' created successfully.\n", directoryName);
}
else {
    perror("mkdir error");
}
```

C. Xây dựng chương trình quản lý thư mục trên bộ nhớ ngoài với các chức năng sau

1. Hàm buildDirectoryTree(const char *rootDir)

   Hàm này thực hiện xây dựng cây thư mcuj từ đầu với gốc cây là rootDir.

2. Hàm `listCurrentFileAndDir(TreeNode *root)`

   Hàm in ra thư mục và các file trong thư mục hiện tại (không duyệt đệ quy xuống các thư mục con ở dưới). Chỉ in mỗi tên file/thư mục trên dòng, không in đường dẫn đầy đủ.

   Xây dựng cây thư mục theo cấu trúc cây được nhập vào từ bàn phím với format dạng.

```
Root: C:\Program Files\Mozilla Firefox\browser
+features
--formautofill@mozilla.org.xpi
--pictureinpicture@mozilla.org.xpi
--screenshots@mozilla.org.xpi
--webcompat-reporter@mozilla.org.xpi
--webcompat@mozilla.org.xpi
-omni.ja
+VisualElements
--PrivateBrowsing_150.png
--PrivateBrowsing_70.png
--VisualElements_150.png
--VisualElements_70.png
```

Trong đó tên các file/ thư mục được in trên 1 dòng. Nếu thư mục thì các dòng đó sẽ có dấu +, còn nếu là file sẽ là dấu -, số lượng dấu + hoặc trừ trước tên file/thư mục là cấp của nó. Như trong đầu vào mẫu thì

- Thư mục gốc hiện tại đang là C:\Program Files\Mozilla Firefox\browser, tên thư mục gốc là browser và đường dẫn đầy đủ sẽ là C:\Program Files\Mozilla Firefox\browser. Khi chạy chương trình thì thư mục gốc sẽ chính là thư mục hiện tại chứa chương trình.
- feature và VisualElements là thư mục con trực tiếp trong thư mục gốc
- omni.ja là file trong thư mục gốc
- formautofill@mozilla.org.xpi là 1 file trong thư mục con features

3. Hàm `listCurrentFileAndDir(TreeNode* currentDirectory, int ASC/DEC)`

   Hàm in ra thư mục và các file trong thư mục hiện tại (không duyệt đệ quy xuống các thư mục con ở dưới). Chỉ in mỗi tên file/thư mục trên dòng, không in đường dẫn đầy đủ.
   Các tên file và thư mục đuoặc sắp theo thứ tự ABC tăng/giảm.

4. Hàm `printTree(TreeNode* node)`

   Hàm dùng để in toàn bộ các file và thư mục trong thư mục hiện tại cũng như các thư mục con của nó ra màn hình.

5. Hàm `printTreeXML(TreeNode* node)`

   Hàm dùng để ghi toàn bộ các file và thư mục trong thư mục hiện tại cũng như các thư mục con của nó ra file XML.

   Chú ý. Các thẻ cấp thấp hơn phải thụt đầu dòng đúng 3 vị trí dấu cách trống (space)

```
<Directory name="browser" path="C:\Program Files\Mozilla Firefox\browser">
    <Directory name="features">
        <File name="formautofill@mozilla.org.xpi"/>
        <File name="pictureinpicture@mozilla.org.xpi"/>
        <File name="screenshots@mozilla.org.xpi"/>
        <File name="webcompat-reporter@mozilla.org.xpi"/>
        <File name="webcompat@mozilla.org.xpi"/>
    </Directory>
    <File name="omni.ja"/>
    <Directory name="VisualElements">
        <File name="PrivateBrowsing_150.png"/>
        <File name="PrivateBrowsing_70.png"/>
        <File name="VisualElements_150.png"/>
        <File name="VisualElements_70.png"/>
    </Directory>
</Directory>
```

6. Hàm `listAllFileOrDir(TreeNode* root, WCHAR pattern)`
   Hàm tìm và in ra toàn bộ các tên file hoặc thư mục có phần tên trùng với từ khóa tìm kiếm.

   Ví dụ nếu đầu vào là

```
? readDirectoryFromInput
Root: C:\Program Files\Mozilla Firefox\browser
+features
--formautofill@mozilla.org.xpi
--pictureinpicture@mozilla.org.xpi
--screenshots@mozilla.org.xpi
```

```
--webcompat-reporter@mozilla.org.xpi
--webcompat@mozilla.org.xpi
+-xpi
-omni.ja
+VisualElements
--PrivateBrowsing_150.png
--PrivateBrowsing_70.png
--VisualElements_150.png
--VisualElements_70.png

và truy vấn là tìm pattern = xpi
Thì kết quả in ra sẽ là
```

[FILE] formautofill@mozilla.org.xpi
[FILE] pictureinpicture@mozilla.org.xpi
[FILE] screenshots@mozilla.org.xpi
[FILE] webcompat-reporter@mozilla.org.xpi
[FILE] webcompat@mozilla.org.xpi
[DIR] xpi

7. Hàm `deleteNode(TreeNode** root, TreeNode* deleteNode)`

   Hàm xóa 1 nút trên cây thư mục hiện tại. Chú ý. Cần giải phóng bộ nhớ của nút bị xóa(và các nút con nếu có)

8. Hàm `addNewDirectoryOrFile(TreeNode* currentRoot, WCHAR name, int type)`

   Hàm tạo 1 thư mục con/file trong thư mục hiện tại.

9. Hàm `moveFileOrDir(TreeNode* currentFileOrDir, TreeNode* newDirectory)`

   Hàm di chuyển 1 file/thư mục tới 1 thư mục khác. Thu mục/file bị di chuyển KHÔNG phải là tổ tiên của thư mục đích tới.

10. Hàm `copyFile(TreeNode* currentFile, TreeNode* newDirectory)`

    Hàm copy 1 file tới 1 thư mục khác.

11. Hàm `copyDirectory(TreeNode* currentDir, TreeNode* newDirectory)`

    Hàm copy 1 thư mục tới 1 thư mục khác.

12. Hàm CheckXML(`TreeNode* node`)

    Hàm này sẽ kiểm tra thư mục hiện tại và nội dung file XML xem có thay đổi gì không (file/thư mục được thêm vào/xóa đi)